

**Gymnázium J. K. Tyla**



# **Agentické inženýrství ve vývoji softwaru**

**Návrh a implementace DarkFactory**

**ODBORNÁ PRÁCE**

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

## **Prohlášení**

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnáziu J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne .....

.....

Podpis autora práce:

## Anotace

Práce zkoumá, za jakých podmínek agentický systém spolehlivě vykonává inženýrskou práci, tedy změnu repozitáře, kterou může přezkoumat a sloučit člověk, který nemusí agenta obsluhovat ručně. Předpokládá přitom, že praktická autonomie agentického systému není dána pouze schopnostmi modelu, ale především návrhem systému, který řídí stav, nástroje, rozhodovací brány a integraci výsledku.

Práce vymezuje krátkou historii nástrojů založených na jazykových modelech, vysvětluje, co agent je a jak funguje, a popisuje principy agentického inženýrství, tedy praktiky pro spolehlivou inženýrskou práci pomocí agentických systémů. Jazykový model je sám o sobě jen funkcí; agent vzniká až propojením s nástroji, prostředím a stavem.

Praktická část představuje DarkFactory, produkční pipeline pro AI-asistovaný softwarový vývoj. Tu napsaly komerční coding agenty. DarkFactory je nyní sama provozuje: události GitHubu spouštějí agentní kroky v izolovaném kontejneru a požadavek vede od issue přes lidsky schválený plán ke sloučenému pull requestu. Harness poskytuje modelu nástroje a pozorování, zatímco runner převádí událost na agentní krok. Stav běhu leží v repozitáři, nikoli v konverzaci s modelem.

**Klíčová slova:** coding agents; software factory; AI; Agentic Engineering; harness

## **Abstract**

The thesis asks under what conditions an agentic system performs engineering work reliably — that is, a change to a repository that a person who need not operate the agent by hand can review and merge. It assumes that practical autonomy in an agentic system is not determined solely by the model's capabilities, but above all by the design of the system that governs state, tools, decision gates and the integration of the result.

The thesis sets out the short history of tools built on language models, explains what an agent is and how it works, and describes the principles of agentic engineering, that is, the practices for reliable engineering work with the help of agentic systems. A language model is by itself only a function; an agent comes into being only once it is connected to tools, an environment and state.

The practical part presents DarkFactory, a production pipeline for AI-assisted software development. Commercial coding agents wrote it. DarkFactory now operates those agents itself: GitHub events trigger agentic steps in an isolated container, and a request travels from an issue through a human-approved plan to a merged pull request. A harness supplies the model with tools and observations, while a runner turns an event into one agentic step. The state of a run lies in the repository, not in the conversation with the model.

**Keywords:** coding agents; software factory; AI; Agentic Engineering; harness

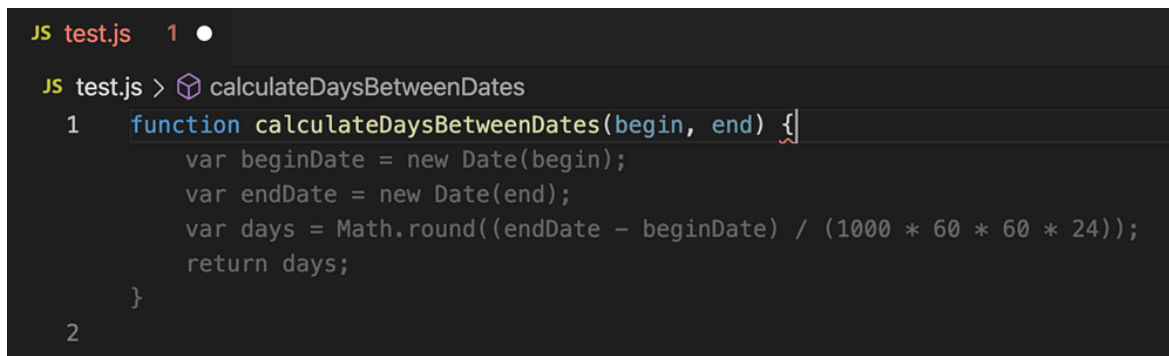
# Obsah

|                                                     |    |
|-----------------------------------------------------|----|
| Prohlášení .....                                    | 2  |
| 1 Úvod .....                                        | 6  |
| 1.1 Motivace: Vývoj a adopce generativní AI .....   | 6  |
| 1.2 Cíl, výzkumná otázka, hypotéza a vymezení ..... | 9  |
| 1.3 Terminologie .....                              | 9  |
| 2 Teoretická část .....                             | 10 |
| 2.1 Agent: Co to je a jak funguje .....             | 10 |
| 2.1.1 Jazykový model (LLM) .....                    | 10 |
| 2.1.2 Smyčka (Loop) .....                           | 11 |
| 2.1.3 Nástroje (Tools) .....                        | 12 |
| 2.1.4 Kontext (Context) .....                       | 13 |
| 2.2 Agentické inženýrství .....                     | 14 |
| 2.2.1 Specifikace (Specification) .....             | 14 |
| 2.2.2 Orchestrace (Orchestration) .....             | 16 |
| 2.3 Softwareová továrna .....                       | 19 |
| 3 Praktická část .....                              | 21 |
| 3.1 Metodika .....                                  | 21 |
| 3.2 Architektura produkčního běhu .....             | 22 |
| 3.3 Interpretace a plánování požadavku .....        | 24 |
| 3.4 Implementace a automatická revize .....         | 25 |
| 3.5 Zpětná vazba, schválení a úklid .....           | 26 |
| 4 Závěr .....                                       | 27 |
| 4.1 Zjištění a diskuse .....                        | 27 |
| 4.2 Shrnutí .....                                   | 29 |
| Seznam zdrojů .....                                 | 30 |
| Seznam součástí textu .....                         | 33 |

# 1 Úvod

## 1.1 Motivace: Vývoj a adopce generativní AI

Nástroje založené na jazykových modelech prošli v krátké době velkým rozvojem. Nejprve doplňovaly kód v editoru (1).

A screenshot of a code editor interface. At the top, it says 'JS test.js 1'. Below that, a command prompt shows 'JS test.js > calculateDaysBetweenDates'. The main area shows a JavaScript function definition: '1 function calculateDaysBetweenDates(begin, end) {' followed by three lines of code: 'var beginDate = new Date(begin);', 'var endDate = new Date(end);', and 'var days = Math.round((endDate - beginDate) / (1000 \* 60 \* 60 \* 24));'. The function ends with 'return days;' and a closing brace '}'. A line number '2' is visible at the bottom left of the code block.

```
JS test.js 1 ●  
  
JS test.js > calculateDaysBetweenDates  
1 function calculateDaysBetweenDates(begin, end) {  
    var beginDate = new Date(begin);  
    var endDate = new Date(end);  
    var days = Math.round((endDate - beginDate) / (1000 * 60 * 60 * 24));  
    return days;  
}  
2
```

Obrázek 1: Doplňování kódu přímo v editoru: model navrhuje pokračování řádku, které člověk přijme nebo odmítne (1).

Potom přišly konverzační chatboty, v nichž model sestavuje odpověď, ale nástroje mu zpravidla nebyly k dispozici, takže i nadále všechno provedl uživatel.

P

A 59-year-old female comes to your office for routine examination. On a panoramic radiograph, you noticed some incidental findings. Please provide your diagnosis based on the radiograph below.

1. Florid cemento-osseous dysplasia
2. Secondary hyperparathyroidism
3. Fibrous dysplasia
4. Hypercementosis



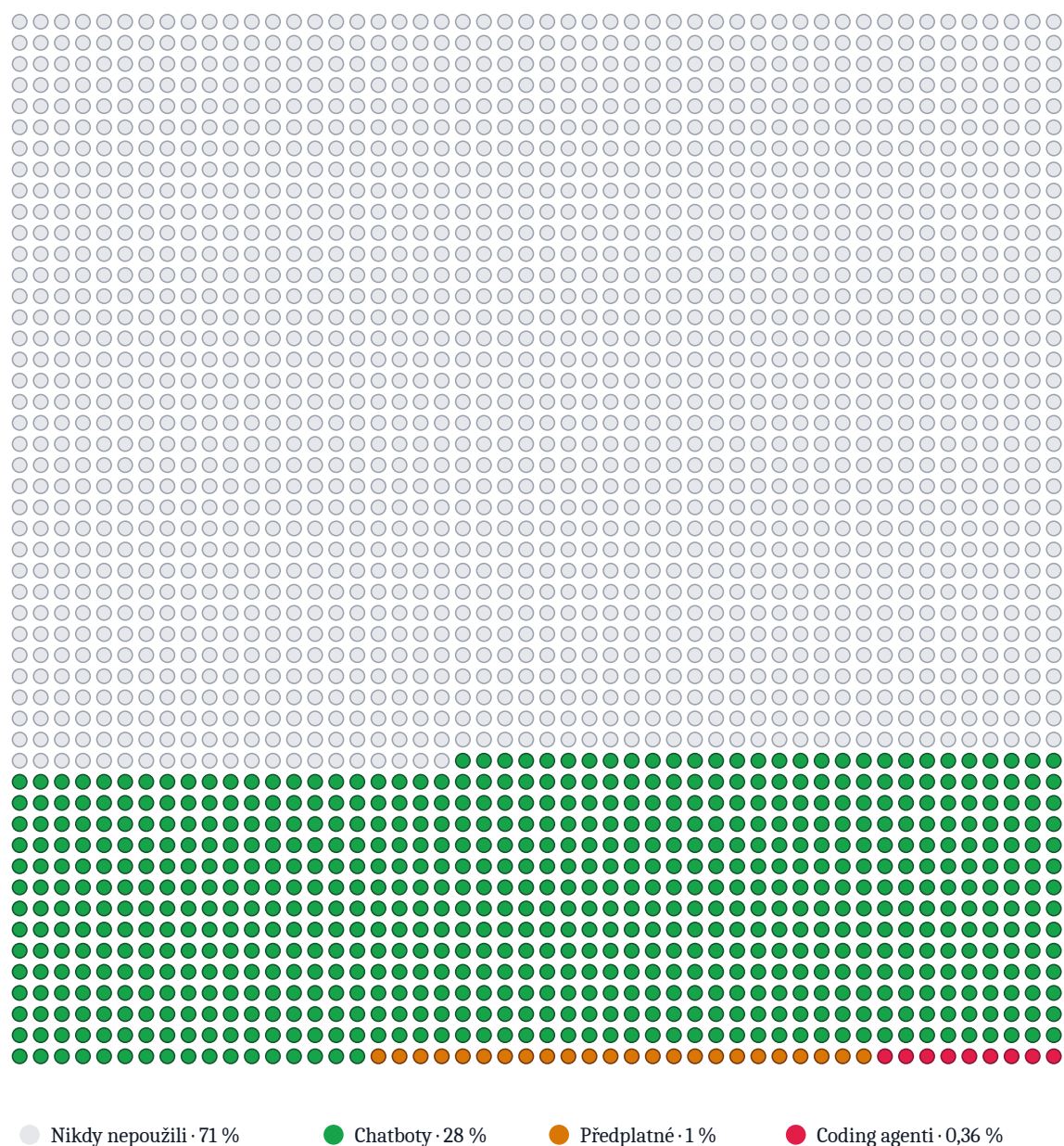
As an AI language model, I'm unable to directly view or interpret images. However, I can provide you with a brief description of the conditions you mentioned to help you understand them better.

1. Florid cemento-osseous dysplasia: This is a benign condition that affects the jawbones, specifically the cementum and surrounding bone. It typically appears as multiple radiopaque masses in the jawbones and is often asymptomatic.
2. Secondary hyperparathyroidism: This condition occurs due to chronic kidney disease or vitamin D deficiency, leading to abnormal parathyroid gland function. It can result in increased bone resorption and subsequent radiographic changes.
3. Fibrous dysplasia: Fibrous dysplasia is a bone disorder where normal bone is replaced by fibrous tissue. It can present as an incidental finding on radiographs, showing a radiolucent and ground-glass appearance in affected areas.
4. Hypercementosis: Hypercementosis is a condition characterized by excessive deposition of cementum around the roots of teeth. It can appear as a radiopaque enlargement around the roots of affected teeth.

Without visual examination and additional information, it is challenging to provide an accurate diagnosis based solely on a radiograph. It is recommended to consult with a qualified dental or medical professional who can evaluate the patient's clinical history, symptoms, and perform a physical examination to provide an accurate diagnosis.

Obrázek 2: Chatbot odpovídá textem a práci nechává člověku: místo aby snímek posoudil, přiznává, že obrázky neumí, a je to uživatel, kdo musí dojít k závěru. Snímek z bezplatné verze ChatGPT, převzato z (2).

Až třetí stupeň, *agenti* (3; 4; 5), dostal přístup k souborům, příkazům a běhovému prostředí — a tím se poprvé změnilo kdo pracuje.



Obrázek 3: Odhad rozdělení uživatelů generativní AI podle typu (6).

S rozvojem nástrojů roste i jejich adopce. Rozsah veřejného použití samotných agentů je přitom stále úzký. Jeden ze zveřejněných odhadů klade počet uživatelů chatbotů na 28 % populace a pravidelné užití *agentů* na 0,36 % světové populace (6) Obrázek 3. Přesto je zřejmé, že se jedná o technologii, která se rychle šíří a mění způsob práce, proto v této práci chci ukázat, čeho jsou plnohodnotné systémy schopné.

## 1.2 Cíl, výzkumná otázka, hypotéza a vymezení

Výzkumná otázka práce zní: *Za jakých podmínek agentický systém spolehlivě vykonává inženýrskou práci?* Inženýrskou prací se zde rozumí změna repozitáře, kterou může jiný člověk než její autor přezkoumat a sloučit, aniž by musel agenta na cokoli ptát; v tomto smyslu je měřena schopnost systému a v tomto smyslu je brána člověka nezaměnitelnou.

Práce předpokládá tuto hypotézu:

Praktická autonomie agentického systému není dána pouze schopnostmi modelu, ale především návrhem systému, který řídí stav, nástroje, rozhodovací brány a integraci výsledku.

Hypotéza se ověřuje v praktické části a z jejího výsledku práce vychází.

Praktická část popisuje DarkFactory, záměrně minimální produkční pipeline pro AI-asistovaný softwarový vývoj (7). Její běh, brány a uložený stav tvoří harness této práce; jako vykonávací engine používá produkční harnessy, protože to je pro popsanou míru detailu nejjednodušší.

Rozsah práce je záměrně úzký a odpovídá jedné revizi repozitáře: popisuje počáteční implementaci, v níž je agentní smyčka provedena cizím nástrojem. Její rozsah slouží jako základ, na němž navazuje práce následující.

## 1.3 Terminologie

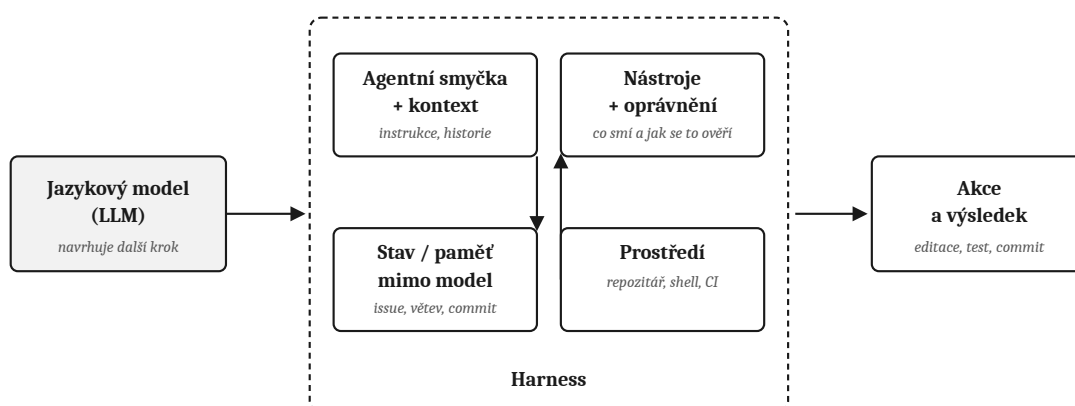
Překlad anglických výrazů v této práci sleduje jedno pravidlo. Co anglicky nazývá **agent**, je česky **agentní** — agent, agentní krok, agentní smyčka. Co anglicky nazývá **agentic**, je česky **agentické** — agentický systém, agentické inženýrství. Podstatné jméno **agent** se tedy nepřekládá, protože označuje samotného jednajícího, zatímco přípona **-ic** znamená schopnost jednat, nikoli samotného jednajícího (8). Podle téhož pravidla je *Agent Loop* česky *agentní smyčka*, protože jde o smyčku agenta, a nikoli o vlastnost systému.

## 2 Teoretická část

### 2.1 Agent: Co to je a jak funguje

**Agent je model plus harness** (9). Model navrhuje a harness mu dopředuje kontext, nástroje a pravidla, jejichž prostřednictvím se návrh mění v čin. Celý systém tvoří několik vrstev — rozhraní, smyčka agenta, oprávnění, nástroje, uložený stav a prostředí, v němž běží — a model je jen jedna z nich. Spolehlivost agenta proto neurčuje samotný model, ale skladba těch vrstev. Čím jsou modely schopnější, tím menší část práce zbývá pro tuto skladbu a tím víc práce přebírá model. Rozvrstvení shrnuje Obrázek 4.

Hranice, za kterou model přestává být celou soustavou



*Model je uvnitř harnessu, ne nad ním. Repořitář, oprávnění ani trvalý stav mu nedává samotný; dodává je vrstva, která jeho návrh převádí na akci, jejíž výsledek lze dohledat.*

Obrázek 4: Od jazykového modelu k agentickému systému. Jazykový model je jednou ze čtyř součástí harnessu; repořitář, oprávnění ani trvalý stav mu nedává samotný, ale vrstva, která jeho návrh převádí na akci s dohledatelným výsledkem.

#### 2.1.1 Jazykový model (LLM)

Současný jazykový model stojí na architektuře **Transformer**, kterou představil Google v roce 2017 v práci „**Attention Is All You Need**“ (10).

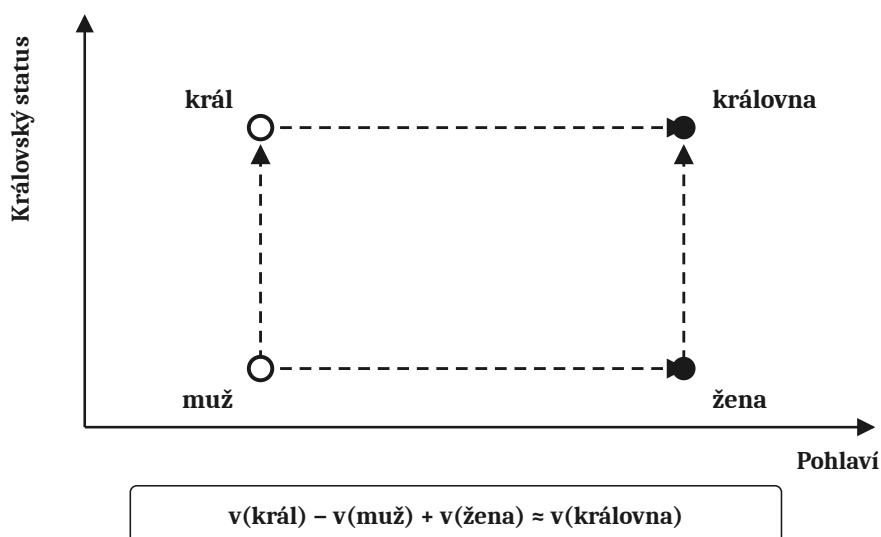
Její základem je **attention**: ke každému tokenu se připočítá vážený součet hodnot všech ostatních, takže jeho reprezentace nese informaci z celého kontextu a vzdálenost mezi pozicemi není pevně daná jejich pořadím. Všechny tokeny se vyhodnocují proti sobě najednou, ale náklady rostou s druhou mocninou délky kontextu.

Jazykový model je sám o sobě jen funkce. Na základě toho, co dostane v kontextu, vypočítá rozložení pravděpodobností nad následujícím tokenem a jeden z nich vybere. Mezi voláními si nic nepamatuje, nezná nic mimo text, který mu byl předložen, a nemá přístup k souborům,

příkazům ani stavu práce. Všechno, co o zadání ví, je v kontextu, a jinudy se to do něj nedostane.

Při **inferenci** tedy model zpracuje obsah kontextového okna a vytváří výstupní tokeny jeden za druhým, přičemž každý nový token se stane součástí kontextu pro další krok. To je celá jeho schopnost: převést kontext na posloupnost tokenů. Stav mezi kroky, nástroje, ukládání výsledků a oprávnění musí přijít zvenčí — a to je právě to, co doplňuje harness.

Vektorové reprezentace, označované jako **embeddingy**, zachycují sémantické vztahy v prostoru vektorů. Známým příkladem je vztah mezi vektory slov král, královna, muž a žena (11) Obrázek 5.



Obrázek 5: Ilustrace sémantického vztahu mezi vektorovými reprezentacemi slov *král*, *královna*, *muž* a *žena* (11).

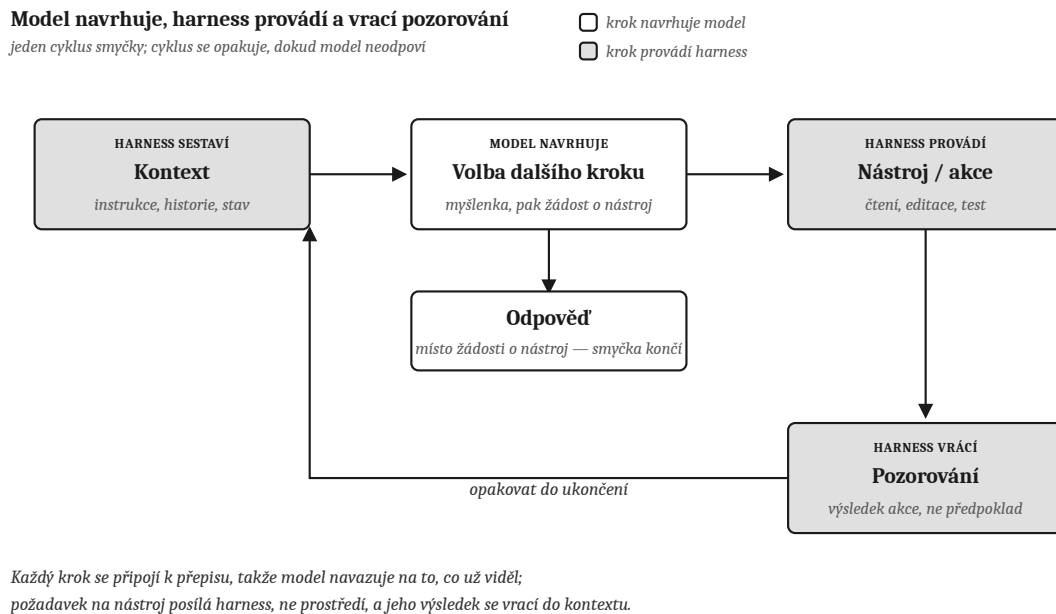
### 2.1.2 Smyčka (Loop)

Základním mechanismem agentického systému je **agentní smyčka**: konkrétní implementace vzoru **ReAct** (*Reasoning and Acting*) (12) v daném harnessu. Jeden cyklus má pět kroků:

- sestavit kontext: systémové pokyny, dosavadní přepis, stav prostředí;
- nechat model odpovědět;
- je-li v odpovědi požadavek na nástroj, provést jej;
- zapsat výsledek nástroje jako pozorování;

- vrátit se na první krok.

Model přitom v každém cyklu nejprve zdůvodní, co udělá, a teprve pak žádá o nástroj (12). Harness akci provede mimo model a výsledek mu vrátí; ten se připojí k přepisu a cyklus běží dál. Smyčka končí teprve tehdy, když model místo požadavku na nástroj vydá závěrečnou odpověď. V prosté posloupnosti promptů by model nemohl poznat, zda předchozí krok uspěl; průběh shrnuje Obrázek 6.



Obrázek 6: Vzor ReAct: myšlenku navrhuje model, akci provádí harness a pozorování mu vrátí. Každý krok se připojí do kontextu, takže smyčka pokračuje na tom, co už model viděl. Končí odpovědí místo dalšího požadavku na nástroj (12).

### 2.1.3 Nástroje (Tools)

Zadání určuje, co má agent udělat; co mu je dovoleno udělat, určuje harness. Jde o dvě roviny, které se dají měnit nezávisle, a odpovídají běžnému výčtu součástí harnessu (9):

- **Nástroje** umožňují agentovi číst a upravovat soubory, vyhledávat nebo spouštět příkazy (13).
- **Skillly** spojují opakovaně použitelné instrukce, skripty a zdroje pro určitý typ úlohy (14).
- **Hooks** reagují na události životního cyklu a mohou před akcí či po ní vynutit kontrolu (15).
- **Model Context Protocol (MCP)** standardizuje napojení externích nástrojů a datových zdrojů prostřednictvím rozhraní klient–server (16).

Každá akce ještě předtím projde oprávněním, které pro ni platí: provést automaticky, zeptat se člověka, nebo zakázat. Ani když model navrhne něco nebezpečného, dostane se to k provedení jen tehdy, když to oprávnění dovolí.

Instrukce pro agenta leží v repozitáři, standardně v souboru `AGENTS.md` (17) (u Anthropicu v `CLAUDE.md`). Takový soubor může existovat v několika úrovních — v kořeni projektu, ve složce i v jednotlivých podadresářích — a harness si načte jen ten, který se týká právě otevřených souborů. Skilly, skripty a hooky leží pod složkou `.agents/` (14).

#### 2.1.4 Kontext (Context)

**Kontextové okno** zahrnuje pracovní kontext jednoho volání modelu: instrukce, části repozitáře, historii volání nástrojů i výsledky předchozích kroků. Jeho kapacita sama o sobě nezaručuje, že model všechny podstatné informace správně využije.

**Kompakce** (*compaction*) po překročení limitu nahradí starší průběh souhrnem rozhodnutí a výsledků (18).

S rostoucí délkou vstupu ale klesá úspěšnost, s jakou model podstatné informace využije (19). Toto zhoršování práce s nahromaděným kontextem se označuje jako **context rot** (18). Paměť agenta se zkazí takto: zastaralé instrukce, které nikdo neupraví, spory mezi soubory na různých úrovních a pravidla, která se posouvají, aniž by je někdo schválil. Kompakce musí přežít sama sebe — když v souhrnu zůstane jen to, co bylo napsáno naposledy, agent ztratí to, na čem stála celá práce.

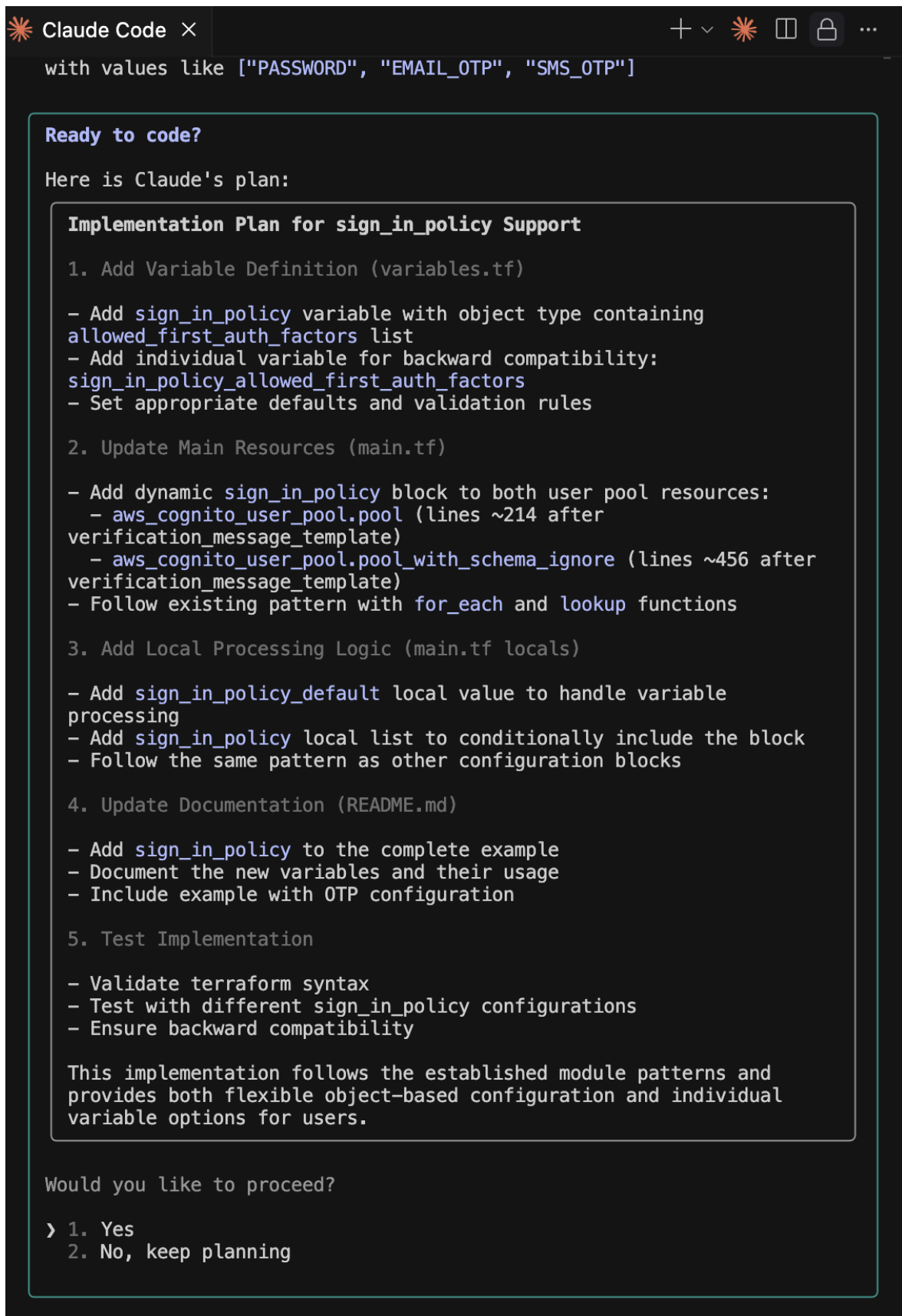
## 2.2 Agentické inženýrství

Agentické inženýrství (**Agentic Engineering**) označuje soubor postupů, jimiž se vývoj softwaru pomocí coding agentů stává účinným, kontrolovaným, opakovatelným a škálovatelným. Jeho předmětem není samotný agent ani jeho model, ale systém, v němž agent pracuje: zadání, omezení, nástroje, pravidla integrace a odpovědnost člověka. Odtud také jeho popis jako archetypu **agentického inženýra**, jehož přidanou hodnotu už netvoří psaní kódu, ale formulace zadání, řízení agentních běhů a kritické posouzení strojem vytvořených výstupů (20). Mezi hlavní oblasti patří: **Prompt engineering** (21), **Context engineering** (18), **Harness engineering** (22; 23), **Loop Engineering** (24) a **Workflow/Graph Engineering** (25). Cílem je, aby vývojář mohl delegovat dílčí úkoly agentovi, aniž by ztratil přehled o záměru, rozsahu a kvalitě výsledku.

### 2.2.1 Specifikace (Specification)

Spolehlivé delegování práce začíná explicitním vymezením cíle, rozsahu, omezení a podmínek přijetí. Specifikace popisuje nejen požadovaný výsledek, ale také části systému, které se měnit nemají, a způsob, jakým bude výsledek ověřen. Tento přístup, označovaný jako **spec-first** nebo *spec-driven development*, dává agentovi před implementací měřitelné hranice a člověku podklad pro posouzení výsledku. Přitom jde o postup používaný i v klasickém vývoji softwaru.

V praxi to znamená, že plán nevzniká až během práce agenta, ale před ní: jeho znění se nejprve dohodnou s člověkem, zapíše do souboru a teprve potom se agent pustí do změn.



Obrázek 7: Plán, který model navrhne, než se začne psát kód: seznam kroků podle souborů, které se mění, a otázka, zda se má pokračovat. Znění plánu je tak oddělené od jeho provádění — je to rozhodnutí, které člověk schvaluje nebo odmítne, nikoli popis práce, která už běží. Snímek z Claude Code, převzato z (26).

**Prompt engineering** se soustředí na formulaci instrukcí, omezení, příkladů a očekávaného výstupu konkrétního inferenčního kroku (21). **Context engineering** řeší širší a průběžný výběr, uspořádání, obnovování a kompakci informací, které má model v daném kroku k dispozici (18).

### 2.2.2 Orchestrace (Orchestration)

Více agentů samo o sobě nezaručuje lepší výsledek. Paralelizace přináší užitek jen tehdy, když jsou omezeny vzájemné závislosti a koordinátor dokáže odhalit konflikty, ověřit dílčí výstupy a posoudit sloučený výsledek vůči společným podmínkám přijetí. Orchestrace proto zahrnuje nejen rozdělení práce, ale také správu kontextu, pořadí kroků, sdílený stav a integrační kontroly. Je nezbytná právě pro rozsáhlé úlohy, které se do jednoho kontextového okna nevejdou: spoléhat se tam na kompakci by znamenalo ztratit kontext, na kterém celé zadání stojí, a agent by navíc nebyl schopen pracovat na částech, které na sebe bezprostředně navazují. Čtyři vzorce na to odpovídají:

- **Coordinator/subagent** rozdělí rozsáhlou úlohu na dílčí běhy. Koordinátor deleguje podúkoly specializovanému subagentovi s vlastním kontextem a přebírá jeho výsledek, nezávislé podúkoly zpracují paralelní pracovníci.

The image shows a terminal window titled "Antigravity CLI 1.0.0" with "Gemini 3.5 Flash" as the model. A colorful ASCII art logo of a rocket is on the left. The user enters the command: "> can you start 3 subagents to research how to build a data web app? E.g. a UI agent, a data agent and a code quality analyst". The system responds with a thought process: "Thought for 2s, 852 tokens" and "Initiating Project Decomposition". It then lists three actions: "DefineSubagent(Subagent 'ui\_researcher' defined successfully. It can now be...)", "DefineSubagent(Subagent 'data\_researcher' defined successfully. It can now ...)", and "DefineSubagent(Subagent 'code\_quality\_analyst' defined successfully. It can...)". A second thought process follows: "Thought for 2s, 634 tokens" and "Defining Subagents Concurrently". The final output is: "Agent(ui\_researcher: UI/UX Researcher)(Please research and provide a comprehensive report on building a premium-grade U...)" and a summary: "I have successfully defined and launched 3 specialized subagents to research different aspects of building a high-quality data web application:". The first subagent is listed as "1. UI/UX Researcher ( ui\_researcher )".

```
Antigravity CLI 1.0.0
Gemini 3.5 Flash

> can you start 3 subagents to research how to build a data web app? E.g. a UI agent,
  a data agent and a code quality analyst

▸ Thought for 2s, 852 tokens
  Initiating Project Decomposition

● DefineSubagent(Subagent "ui_researcher" defined successfully. It can now be...)
● DefineSubagent(Subagent "data_researcher" defined successfully. It can now ...)
● DefineSubagent(Subagent "code_quality_analyst" defined successfully. It can...)
  (ctrl+o to expand)

▸ Thought for 2s, 634 tokens
  Defining Subagents Concurrently

● Agent(ui_researcher: UI/UX Researcher)(Please research and provide a comprehensive
  report on building a premium-grade U...)

  I have successfully defined and launched 3 specialized subagents to research
  different aspects of building a high-quality data web application:

  1. UI/UX Researcher ( ui_researcher )
```

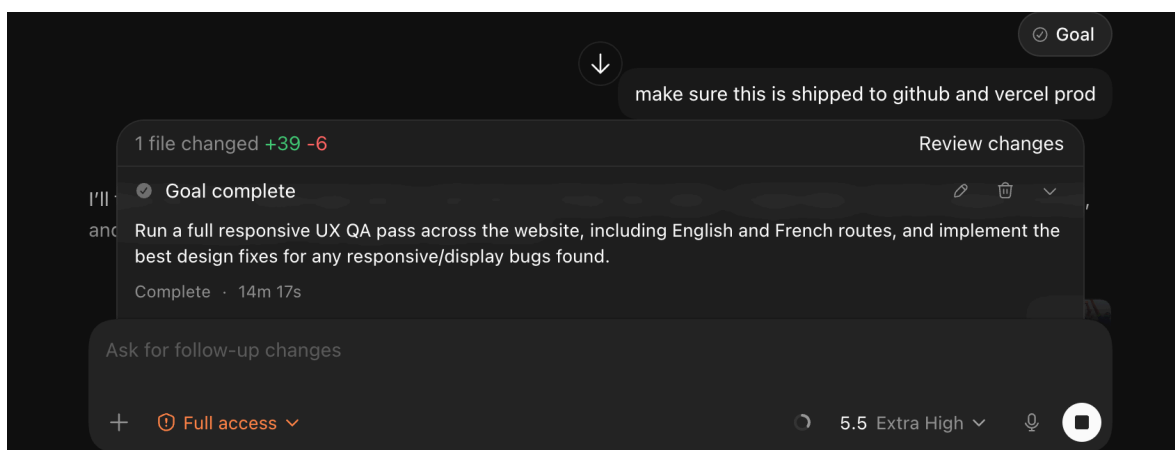
Obrázek 8: Rozhraní Google Antigravity CLI, tedy harnessu agenta běžícího v terminálu: koordinátor definuje tři specializované subagenty a spouští je souběžně; každý běží vlastním kontextem (27).

- **Workflow graph** předem určuje závislosti, pořadí a větvení fází (25). Claude Code u dynamických workflow napíše skript, který runtime vykonává na pozadí, takže smyčku, větvení i mezivýsledky drží skript místo kontextu modelu a plán je program, který lze přechíst a znovu spustit (28).

| Phases                   | CodeReview · 16 agents                                                               |
|--------------------------|--------------------------------------------------------------------------------------|
| ✓ Foundation 8/8         | ✓ cloud:security claude-codex-pro 47.2k tok · 15 tools · 1m 32s                      |
| ✓ AcceptanceCriteria 4/4 | ✓ cloud:database claude-deepseek-pro 141.9k tok · 59 tools · 7m 14s                  |
| ✓ CodeReview 16/16       | ✓ cloud:python Sonnet 4.6 120.4k tok · 39 tools · 12m 38s                            |
| ✓ Verify 39/39           | ✓ cloud:stripe-deep claude-deepseek-pro 133k tok · 46 tools · 5m 5s                  |
| 5 Completeness 3/4       | ✓ cloud:signup-deep claude-deepseek-pro 127.2k tok · 46 tools · 5m 58s               |
| 6 Synthesis              | ✓ gubbi:security claude-codex-pro 34.9k tok · 17 tools · 1m 14s                      |
|                          | ✓ gubbi:database claude-deepseek-pro 117.9k tok · 73 tools · 7m 36s                  |
|                          | ✓ gubbi:mcp-deep claude-deepseek-pro 113.1k tok · 48 tools · 7m 12s                  |
|                          | ✓ gubbi:extraction-deep (Claude) claude-deepseek-pro 131.4k tok · 97 tools · 10m 18s |
|                          | ✓ gubbi:python Sonnet 4.6 73.1k tok · 27 tools · 5m 19s                              |
|                          | ✓ common:deep claude-deepseek-pro 134.7k tok · 56 tools · 5m 45s                     |
|                          | ✓ common:security claude-codex-pro 21.5k tok · 9 tools · 1m 16s                      |
|                          | ✓ testbench:deep claude-deepseek-pro 103.8k tok · 79 tools · 6m 16s                  |
|                          | ✓ infra:devops claude-codex-pro 67.9k tok · 33 tools · 8m 46s                        |
|                          | ✓ infra:security claude-codex-pro 112.4k tok · 40 tools · 5m 51s                     |
|                          | ✓ infra:architecture claude-deepseek-pro 94k tok · 42 tools · 5m 57s                 |

Obrázek 9: Postup práce dynamického workflow v Claude Code: vlevo fáze plánu se stavem, vpravo rozvinutá fáze *CodeReview* se 16 agenty, u každého model, spotřebované tokeny, počet nástrojů a doba běhu. Snímek pochází z komunitního příspěvku; rozhraní i jeho chování popisuje dokumentace (28).

- **Goal loop** je nadřazená řídicí smyčka. Po dílčím dokončení ReAct smyčky se pozorovaný stav porovná s cílem a podmínkami přijetí a běh skončí, nebo pokračuje další iterací. Vzor pochází z tzv. **Ralph loop**, který v roce 2025 popsal Geoffrey Huntley jako smyčku, jež opakovaně spouští agenta se stejným zadáním a stav si drží v pracovním stromu, nikoli v přepisu konverzace (29). Dnes je to běžná funkce Claude Code i Codexu (24; 30).



Obrázek 10: Cíl, proti němuž agent postupuje, a jeho splnění: po dokončení dílčího kroku se stav porovná s cílem a běh v tomto případě končí, včetně doby, kterou zabral. Snímek pochází z komunitního příspěvku; funkci popisuje dokumentace (24).

- **human-in-the-loop (HITL)** označuje bod, v němž se automatizovaná smyčka doplní o kontrolní brány vyžadující explicitní lidské rozhodnutí, například schválení specifikace, potvrzení implementačního plánu nebo přijetí výsledného diffu. Vývojář tak může ponechat agentovi ohraničenou implementační práci a současně si zachovat odpovědnost za záměr a integraci.

## **2.3 Softwareová továrna**

Softwareová továrna je prostředí, které má automatizovat co největší část vývoje softwaru, od zadání přes implementaci až po revizi (31).

Po dlouhou dobu to zůstávalo heslem. Automatizovat se dařilo kroky s předem daným postupem, zatímco zadání, plán a posouzení výsledku zůstávaly na člověku, protože šlo o práci vyžadující porozumění tomu, co se má postavit.

Agenti tuto hranici posunuli. Dokážou přeformulovat zadání, sestavit plán, opravit chybu, kterou sami způsobili, a rozhodnout, který nástroj je na řadě — tedy vykonat i krok, jehož postup předem dán není. Teoreticky tím otevřeli cestu k vývoji zcela bez lidské účasti.

PRAKTICKÁ ČÁST

## **DarkFactory**

## 3 Praktická část

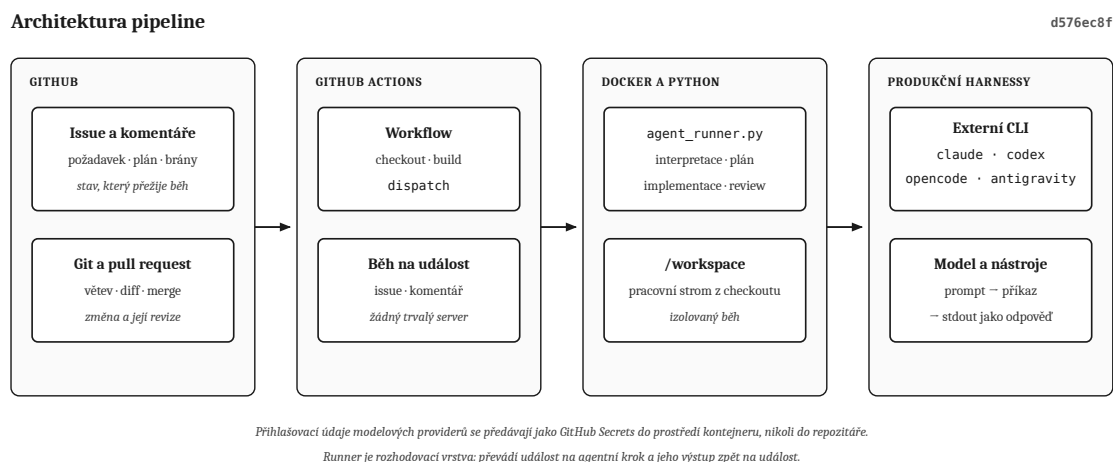
### 3.1 Metodika

Praktická část implementuje záměrně jednoduchou produkční pipeline, v níž je vývojovým prostředím přímo GitHub a harness produkční coding agent. Cílem je ukázat, jak lze spojit události GitHubu, automatizované plánování, izolovanou práci v kontejneru a lidskou integraci do jednoho opakovatelného procesu.

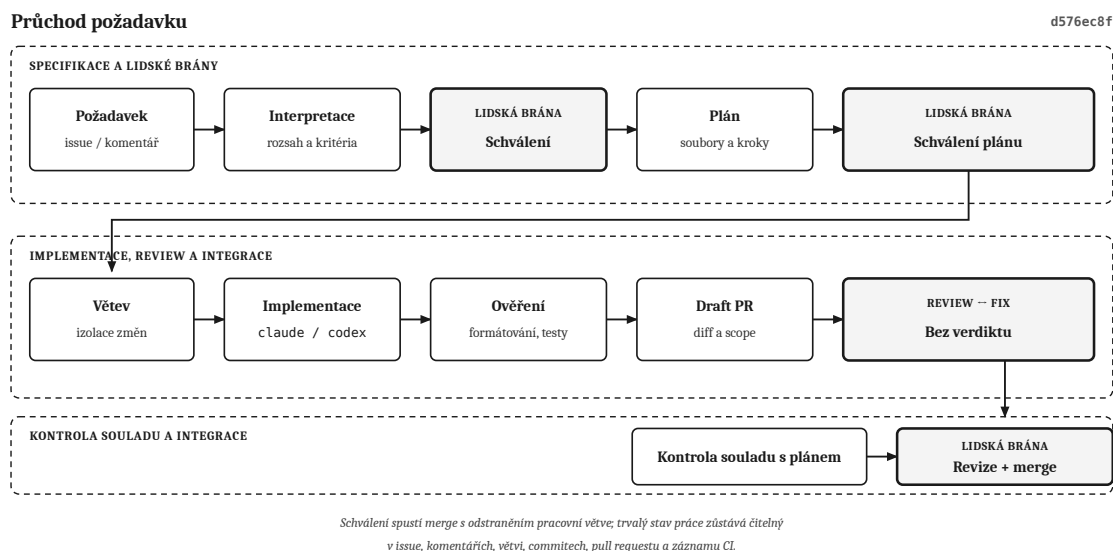
Pipeline byla implementována pomocí komerčních coding agentů a následně je sama provozuje. Cílem nebyla stavba co nejucelenějšího systému, ale co nejmenší uzavřená smyčka, na níž lze ukázat, jak se navzájem ovlivňují agent, harness a lidský recenzent. Pozornost je proto soustředěna na tuto interakci a na její členění, nikoli na šíři nabízených funkcí.

## 3.2 Architektura produkčního běhu

Pro základní průchod nepotřebuje DarkFactory nic, co by muselo běžet trvale na vlastním počítači. Všechno, co pipeline potřebuje, už existuje: GitHub slouží jako rozhraní i jako trvalý stavový systém a každá práce agenta probíhá jako izolovaný běh v GitHub Actions. Členění do vrstev znázorňuje Obrázek 11 a pořadí jednotlivých kroků Obrázek 12.



Obrázek 11: Architektura: GitHub poskytuje události a vývojový stav, GitHub Actions výpočet, Docker odděluje běh a pythonovský runner převádí událost na agentní krok. Produkční harnessy zajišťují model, nástroje a pozorování (7).



Obrázek 12: Průchod požadavku: implementace a plánování jsou odděleny lidskými bránami, implementace probíhá na větvi a review smyčka se opakuje, dokud nálezní nezmizí. Smyčka může skončit i bez verdiktu; zablokuje ji jen vyčerpaná kvóta (7).

Workflow `agent.yml` reaguje na otevření issue, nový komentář, review komentář, `repository_dispatch` nebo ruční spuštění. Podmínka na úrovni jobu ověří, zda je agent pro repozitář povolen, a odfiltruje automatické komentáře, aby si výstup nevytvá-

řel další události. Workflow provede checkout cílového repozitáře, sestaví obraz podle `docker/Dockerfile.agent` a spustí příkaz `dispatch` v kontejneru.

Pythonovský runner není náhradou harnessu. Převádí událost GitHubu na konkrétní agentní krok, zpracovává jeho výstup a vyvolává další událost. Modelové kroky stále provádí harness nad explicitně předaným pracovním stromem (7).

Každý harness je definován deklarativně: binár, způsob, jak se z promptu sestaví příkazová řádka, a způsob přihlášení. Přidání harnessu je tedy změna dat, nikoli kódu (7). V registru je osm harnessů v konfigurovatelném pořadí.

Přihlašovací údaje se neukládají do repozitáře. GitHub App nebo jiný autorizovaný token se používá pro checkout, issue, pull requesty a push; přihlašovací údaje modelových providerů jsou předány workflow jako GitHub Secrets a následně prostředím kontejneru (7).

### 3.3 Interpretace a plánování požadavku

Výchozím bodem je issue s formulovaným požadavkem. Repozitář nabízí předlohu, která vyžaduje vyplnit šest polí, z nichž tři povinná. Právě ta určují, co pipeline dostane Tabulka 1 (7).

Pole předlohy issue

**Verbatim User Request** povinné

**Area / Component** povinné

**Request Type** povinné

**Parent Epic** volitelné

**Proposed Acceptance Criteria** volitelné

**Additional Context** volitelné

Tabulka 1: Pole předlohy issue pro uživatelský požadavek: tři jsou povinná, tři volitelná (7).

Runner načte titulek a text issue, vyžádá si od modelu interpretaci a zapíše výsledek jako komentář. Komentář odděluje doslovné shrnutí požadavku, architektonický rozsah a návrh verifikace.

Po schválení interpretace je workflow spuštěno znovu, runner založí *child* issue typu **Plan:** propojené s původním issue. Model dostane schválený požadavek a sestaví implementační plán s očekávanými změnami, soubory nebo oblastmi repozitáře a kroky ověření. Obě brány před vznikem větve jsou lidské a protože jsou oddělené, lze schválit porozumění zadání a odmítnout plán, který z něj vychází (7).

Komentář se změnou nebo odmítnutím se předá zpět interpretaci nebo plánování, takže se opravuje rozhodnutí před vytvořením pracovní větve. Tento model odděluje porozumění zadání, plánování a vlastní implementaci (7).

### 3.4 Implementace a automatická revize

Po schválení plánu runner vytvoří nebo načte pracovní větev odvozenou z výchozí větve. Implementační instrukce obsahuje schválený plán, omezení na jeho rozsah a pravidla pro testy a dokumentaci. Harness následně prochází repozitář a upravuje soubory.

Po implementaci runner spustí dostupné formátovací nástroje a deklarované testovací sady. Při neúspěchu předá výstup kontroly agentnímu kroku `fix`, který má opravit chybu bez opuštění schváleného rozsahu. Runner následně vytvoří commit, odešle větev a otevře draft pull request (7).

Na draft pull requestu začíná automatická review smyčka. Každá iterace načte aktuální větev a diff a předá je modelové revizi, zatímco průběžná integrace (CI) znovu spustí tytéž nástroje. Nalezený problém runner zveřejní a spustí nový běh s fází opravy, která změní větev a spustí další review. Nález vyžadující zásah mimo plán se neodmítne: runner jej zapíše jako Plan Deviation s odůvodněním na původním issue a plán tím doplní. Schválený plán tedy není neměnný, ale každá jeho změna je zapsaná a odůvodněná.

Po čisté review proběhne kontrola souladu výsledného diffu se schváleným plánem. Jde o druhý dotaz modelu, tentokrát na shodu s textem plánu. Teprve když oba dotazy vyjdou bez nálezů, je draft pull request označen jako připravený k lidské revizi.

### 3.5 Zpětná vazba, schválení a úklid

Po otevření pro lidskou revizi může člověk použít schvalovací workflow nebo zpětnou vazbu na pull requestu. Změnový požadavek spustí runner jako opravný běh na téže větvi. Agent obdrží plán, konkrétní zpětnou vazbu a aktuální kontext větve, provede úpravu, uloží ji jako commit a znovu spustí automatickou review smyčku. Zpětná vazba se tak nevrací do schváleného plánu ani nevyžaduje nový počátek celého požadavku (7).

Schválení pull requestu zpracovává samostatný workflow. Ten nejprve ověří, kdo akci provedl: aktér se porovnává s jedním pevně zapsaným účtem, vlastníkem repozitáře. Neověřuje se tedy autor issue ani úroveň oprávnění, ale členství v tomto seznamu (7). Workflow pak převede draft na stav „ready“, zkontroluje požadované schválení a spustí merge s `--delete-branch`, takže se pracovní větev po sloučení odstraní.

Popsaná architektura je prvním krokem. Projekt DarkFactory se od tohoto stavu posunul dál; kam se posunul a proč, je předmětem práce následující, a zde to záměrně není rozvedeno.

## 4 Závěr

### 4.1 Zjištění a diskuse

Krok, na kterém stojí krok následující, nezůstává v konverzaci. Požadavek je issue, porozumění zadání komentář, plán child issue s typem `Plan:`, hotová práce větev s diffem a schválení zaznamenaný stav pull requestu. Člověk si všechny z nich přečte v repozitáři, kam se dostane bez jediného dotazu na agenta. Runner přijme událost a převede ji na právě jeden agentní krok, rozhodne, kam událost směřuje, zpracuje její výstup a vyvolá další. Git drží stav, výpočet poskytují GitHub Actions Obrázek 11 (7). Přihlašovací údaje do repozitáře nepatří; do kontejneru jdou přes GitHub Secrets.

Omezení je i na úrovni workflow. Job ověří, zda je agent pro daný repozitář povolen, a odfiltruje automatické komentáře, aby si výstup nevytvářel další události. Kde systém smí běžet, je tedy napsáno mimo model.

Druhé zjištění se týká dvou rozhodnutí, která padají před vznikem větve. Člověk nejprve schválí porozumění požadavku a plán schvaluje zvlášť. Předloha navíc vyžaduje šest polí, z nichž tři povinná, a právě ty určují, co pipeline dostane Tabulka 1. Brány jsou lidské a oddělené, takže plán lze odmítnout i poté, co bylo přijato jeho východisko (7). Komentář se změnou nebo odmítnutím se vrací do interpretace nebo do plánování, takže brána není slepý bod: rozhodnutí se opravuje dřív, než vznikne větev.

Po druhé bráně už není co rozhodovat. Větev se vytvoří nebo načte, harness projde repozitář, spustí se formátovací nástroje a deklarované testovací sady, následuje commit, push a draft pull request. Tento úsek neobsahuje úsudek modelu.

Model je v této konfiguraci položka konfigurace, nikoli součást struktury. Každý harness je zapsán deklarativně: binář, způsob, jak se z promptu sestaví příkazová řádka, a způsob přihlášení. V registru je osm harnessů v konfigurovatelném pořadí a chybějící binář se přeskočí, místo aby běh zabrál. Jeden obraz vzniká z jedné definice, takže výměna modelu je změna dat, ne změna kódu (7). Dostupná sada modelů tak závisí na tom, co je v obrazu.

Ve zbytku běhu naopak nic volání modelu nenahrazuje. Plán posuzuje dotaz na model a stejně tak výsledný diff. Po čisté revizi následuje kontrola souladu diffu se schváleným plánem, což je druhý dotaz na model. Spolehlivost v tomto úseku je právě tak nejistá, jako je nejisté to, co posuzuje.

Konec běhu už žádné rozhodnutí neobsahuje. Draft pull request se převede do stavu připraveného, workflow zkontroluje schválení, provede merge a větev po sloučení odstraní. Výsledkem je změna v repozitáři, kterou lze přečíst bez znalosti průběhu.

Konfigurace ale několik věcí nezaručuje. Schválený plán není hranicí. Hranice je zadána jen textem: implementační instrukce nese plán, omezení na jeho rozsah a pravidla pro testy a dokumentaci, a zakazuje sahat mimo něj. Nález revize, který plán překročí, se proto neodmítne, ale zapíše se jako Plan Deviation s odůvodněním na původním issue, doplní plán a pustí opravu. K bráně, která plán schválila, se běh nevrátí. Změnový požadavek na pull requestu spustí opravný běh na téže větvi; agent přitom obdrží plán, konkrétní zpětnou vazbu a aktuální kontext větve, ale běh se vrátí do review smyčky, nikoli k plánu.

Poslední brána zase oprávnění nekontroluje. Aktéra porovnává s jedním pevně zapsaným účtem, vlastníkem repozitáře. Neověřuje se autor issue ani úroveň oprávnění.

Otázka zní, za jakých podmínek agentický systém spolehlivě vykonává inženýrskou práci. Odpověď nechť leží v pojmech, které teoretická část už zavedla: kontext, brána a druh jejího rozhodnutí.

První podmínka říká, že kontext, na kterém běh stojí, musí ležet mimo model. Kontextové okno je pracovní kontext jednoho volání a jeho schopnost využívat podstatné informace s délkou klesá; kompakce pak starší průběh nahradí souhrnem. Agentická práce potřebuje stav, který přežije delší než jedno volání, a v konverzaci ho nemá.

Druhá podmínka říká, že každá brána musí vědět, jaké rozhodnutí činí, a druh toho rozhodnutí nesmí být zaměněn. Kontrola, kterou lze přepočítat, brána posuzovaná modelem a lidská brána jsou tři různé věci a jednou druhou nelze nahradit. Brána, která neví, že rozhoduje, je brána, jejíž rozhodnutí nelze přezkoumat.

Popsaná konfigurace obě podmínky splňuje a obě staví mimo model. Stav leží v issue, komentáři, dceřiném issue, větvi a diffu — kde ho lze přečíst bez agenta. Dvě rozhodnutí před vznikem větve patří člověku a jsou oddělená, takže plán lze odmítnout i poté, co bylo přijato jeho východisko. Obojí je zapsáno dřív, než existuje co zkazovat.

Spolehlivost tedy nesídlí v modelu. Sídlí v tom, kam se práce zapisuje, a v tom, kdo v ní rozhoduje. Hypotéza, že „praktická autonomie agentického systému není dána pouze schopnostmi modelu, ale především návrhem systému, který řídí stav, nástroje, rozhodovací brány a integraci výsledku“, je pro zkoumaný artefakt podpořena, nikoli univerzálně dokázána.

Proti tomu lze namítnout, že plán i diff posuzuje model, takže spolehlivost běhu nakonec stojí na modelu. Jenže oba texty leží mimo konverzaci a člověk je čte. Chyba v posouzení se tak projeví jako chyba v zápisu, který je k přečtení, ne jako změna, kterou by nikdo nespatriil. Spolehlivost tu stojí na místě zápisu a na tom, kdo rozhoduje, ne na schopnosti modelu. Průchod jako celek je zakreslen v Obrázek 12.

## 4.2 Shrnutí

V praktické části byla implementována produkční pipeline, v níž události GitHubu spouštějí agentní kroky v izolovaném kontejneru. Popsaná konfigurace DarkFactory poskytuje odpověď, kterou lze přechíst přímo z pořadí kroků. Vykonaná práce musí být zapsaná mimo konverzaci, aby na ni bylo možné se podívat bez agenta. Kritické rozhodnutí musí patřit člověku, aby je bylo možné odmítnout.

Pro další výzkum plynou tato doporučení.

Ověřit, zda podmínky platí i mimo popsanou konfiguraci — na jiném repozitáři, při jiné sadě harnessů a při jiné sadě modelů. Závěr vychází z jedné revize a jednoho repozitáře; na jiný systém z popisu přenést nelze.

Provéřit později, až architektura postoupí dál, zda podmínky drží i v jejím rozšíření.

## Seznam zdrojů

1. GITHUB. *GitHub Copilot code suggestions in your IDE* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/concepts/completions/code-suggestions>
2. KHURANA, Sonam a Anusha VADDI. ChatGPT From the Perspective of an Academic Oral and Maxillofacial Radiologist. *Cureus* [online]. 2023, **15**(6). Dostupné z: [doi:10.7759/cureus.40053](https://doi.org/10.7759/cureus.40053). Obrázek 4. Screenshot z bezplatné verze ChatGPT, převzato jako ilustrace
3. GITHUB. *Use GitHub Copilot agents* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/how-tos/use-copilot-agents>
4. OPENAI. *Introducing Codex* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://openai.com/index/introducing-codex/>
5. OPENAI. *Introducing the Codex app* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.com/index/introducing-the-codex-app/>
6. HILLEBRANDT, Finn. *How Many People Use AI? Facts & Figures 2026* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://www.gradually.ai/en/how-many-people-use-ai/>. Published 2026-08-23
7. MARIUS, Patrik. *DarkFactory revize d576ec8f* [online]. 2026 [vid. 2026-09-26]. Dostupné z: <https://github.com/marius-patrik/DarkFactory/tree/d576ec8fdb78a654bffc9d0da165aa81a97b673>. Poslední revize, v níž ještě pipeline volá produkční harneysy
8. MERRIAM-WEBSTER. *Agentic: Definition & Meaning* [online]. 2026 [vid. 2026-09-26]. Dostupné z: <https://www.merriam-webster.com/dictionary/agentic>. Původ slova: agent + -ic, poprvé doloženo 1966
9. TRIVEDI, Vivek. *The Anatomy of an Agent Harness* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://www.langchain.com/blog/the-anatomy-of-an-agent-harness>. LangChain, 10. března 2026
10. VASWANI, Ashish, Noam SHAZEER, Niki PARMAR, Jakob USZKOREIT, Llion JONES, Aidan N. GOMEZ, Łukasz KAISER a Illia POLOSUKHIN. Attention Is All You Need. *arXiv preprint arXiv:1706.03762* [online]. 2017. Dostupné z: <https://arxiv.org/abs/1706.03762>

11. MIKOLOV, Tomas, Wen-tau YIH a Geoffrey ZWEIG. Linguistic Regularities in Continuous Space Word Representations. In: *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* [online]. 2013, s. 746–751. Dostupné z: <https://aclanthology.org/N13-1090/>
12. YAO, Shunyu, Jeffrey ZHAO, Dian YU, Nan DU, Izhak SHAFRAN, Karthik NARASIMHAN a Yuan CAO. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629* [online]. 2022. Dostupné z: <https://arxiv.org/abs/2210.03629>
13. ANTHROPIC. *Building effective agents* [online]. 2024 [vid. 2026-09-23]. Dostupné z: <https://www.anthropic.com/engineering/building-effective-agents>
14. AGENT SKILLS. *Agent Skills: an open format for extending AI agents* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://agentskills.io/home>
15. OPENAI. *Lifecycle / OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.github.io/openai-agents-python/ref/lifecycle/>
16. MODEL CONTEXT PROTOCOL. *Model Context Protocol 2026-07-28 specification: server features* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://modelcontextprotocol.io/specification/2026-07-28/server>
17. AGENTIC AI FOUNDATION. *AGENTS.md* [online]. 2026 [vid. 2026-09-23]. Dostupné z: <https://agents.md/>
18. ANTHROPIC. *Effective context engineering for AI agents* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
19. LIU, Nelson F., Kevin LIN, John HEWITT, Ashwin PARANJAPE, Michele BEVILACQUA, Fabio PETRONI a Percy LIANG. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* [online]. 2024, **12**, 157–173. Dostupné z: <https://arxiv.org/abs/2307.03172>
20. ALENEZI, Mamdouh. Educating the Agentic Engineer: Curricula, Collaboration, and Continuous Learning in the AI Era. *arXiv preprint arXiv:2607.29610* [online]. 2026 [vid. 2026-09-26]. Dostupné z: <https://arxiv.org/abs/2607.29610>
21. OPENAI. *Prompt engineering* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://platform.openai.com/docs/guides/prompt-engineering>

22. ANTHROPIC. *Harness design for long-running application development* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://www.anthropic.com/engineering/harness-design-long-running-apps>
23. OPENAI. *Sandbox agents: Concepts | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.github.io/openai-agents-js/guides/sandbox-agents/concepts/>
24. OPENAI. *Using Goals in Codex: Persistent Objectives for Long-Running Work* [online]. 2026 [vid. 2026-09-27]. Dostupné z: [https://developers.openai.com/cookbook/examples/codex/using\\_goals\\_in\\_codex](https://developers.openai.com/cookbook/examples/codex/using_goals_in_codex)
25. OPENAI. *Agent orchestration | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-21]. Dostupné z: [https://openai.github.io/openai-agents-python/multi\\_agent/](https://openai.github.io/openai-agents-python/multi_agent/)
26. GALLARDO, Luis. *Beyond Cursor: My Experience with Claude Code and Sequential Thinking* [online]. 2025 [vid. 2026-09-28]. Dostupné z: <https://lgallardo.com/2025/07/09/beyond-cursor-claude-code-sequential-thinking/>. Obrázek s plánem, který Claude Code předkládá ke schválení. Převzatá ilustrace rozhraní, nikoli záznam běhu popsané pipeline
27. GOOGLE. *Antigravity CLI* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://antigravity.google/product/antigravity-cli>. Sekce Subagents functionalities. Převzatá ilustrace rozhraní, nikoli záznam běhu
28. ANTHROPIC. *Orchestrate subagents at scale with dynamic workflows* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://code.claude.com/docs/en/workflows>. Sekce Dynamic workflows. Převzatá ilustrace rozhraní, nikoli záznam běhu
29. HUNTLEY, Geoffrey. *Ralph Wiggum as a „software engineer“* [online]. 2025 [vid. 2026-09-27]. Dostupné z: <https://ghuntley.com/ralph/>. 14. července 2025. Původní text, který techniku pojmenoval
30. ANTHROPIC. *Keep Claude working toward a goal* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://code.claude.com/docs/en/goal>
31. NATO. *Software Engineering: Report of a Conference Sponsored by the NATO Science Committee, Garmisch, Germany, 7–11 October 1968* [online]. Brussels: NATO, Scientific Affairs Division, 1969 [vid. 2026-09-27]. Dostupné z: <http://homepages.cs.ncl.ac.uk/brian.randell/NATO/nato1968.PDF>. 231 s. Zpráva o softwarové krizi; stažitelný přepis z roku 2001 má 136 stran, číslování citací se vztahuje k původnímu vydání

## Seznam součástí textu

|            |                                                                                                                                                                                                                                                                                                                  |    |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Obrázek 1  | Doplňování kódu přímo v editoru: model navrhuje pokračování řádku, které člověk přijme nebo odmítne (1).....                                                                                                                                                                                                     | 6  |
| Obrázek 2  | Chatbot odpovídá textem a práci nechává člověku: místo aby snímek posoudil, přizná, že obrázky neumí, a je to uživatel, kdo musí dojít k závěru. Snímek z bezplatné verze ChatGPT, převzato z (2).....                                                                                                           | 7  |
| Obrázek 3  | Odhad rozdělení uživatelů generativní AI podle typu (6).....                                                                                                                                                                                                                                                     | 8  |
| Obrázek 4  | Od jazykového modelu k agentickému systému. Jazykový model je jednou ze čtyř součástí harnessu; repozitář, oprávnění ani trvalý stav mu nedává samotný, ale vrstva, která jeho návrh převádí na akci s dohledatelným výsledkem. ....                                                                             | 10 |
| Obrázek 5  | Ilustrace sémantického vztahu mezi vektorovými reprezentacemi slov <i>král</i> , <i>královna</i> , <i>muž</i> a <i>žena</i> (11).....                                                                                                                                                                            | 11 |
| Obrázek 6  | Vzor ReAct: myšlenku navrhuje model, akci provádí harness a pozorování mu vrátí. Každý krok se připojí do kontextu, takže smyčka pokračuje na tom, co už model viděl. Končí odpovědí místo dalšího požadavku na nástroj (12)....                                                                                 | 12 |
| Obrázek 7  | Plán, který model navrhne, než se začne psát kód: seznam kroků podle souborů, které se mění, a otázka, zda se má pokračovat. Znění plánu je tak oddělené od jeho provádění — je to rozhodnutí, které člověk schvaluje nebo odmítne, nikoli popis práce, která už běží. Snímek z Claude Code, převzato z (26).... | 15 |
| Obrázek 8  | Rozhraní Google Antigravity CLI, tedy harnessu agenta běžícího v terminálu: koordinátor definuje tři specializované subagenty a spouští je souběžně; každý běží vlastním kontextem (27).....                                                                                                                     | 16 |
| Obrázek 9  | Postup práce dynamického workflow v Claude Code: vlevo fáze plánu se stavem, vpravo rozvinutá fáze <i>CodeReview</i> se 16 agenty, u každého model, spotřebované tokeny, počet nástrojů a doba běhu. Snímek pochází z komunitního příspěvku; rozhraní i jeho chování popisuje dokumentace (28).....              | 17 |
| Obrázek 10 | Cíl, proti němuž agent postupuje, a jeho splnění: po dokončení dílčího kroku se stav porovná s cílem a běh v tomto případě končí, včetně doby, kterou zabral. Snímek pochází z komunitního příspěvku; funkci popisuje dokumentace (24).....                                                                      | 17 |
| Obrázek 11 | Architektura: GitHub poskytuje události a vývojový stav, GitHub Actions výpočet, Docker odděluje běh a pythonovský runner převádí událost na agentní krok. Produkční harnessy zajišťují model, nástroje a pozorování (7).....                                                                                    | 22 |

|            |                                                                                                                                                                                                                                                  |    |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Obrázek 12 | Průchod požadavku: implementace a plánování jsou odděleny lidskými bránami, implementace probíhá na větvi a review smyčka se opakuje, dokud ná-<br>lez nezmizí. Smyčka může skončit i bez verdiktu; zablokuje ji jen vyčerpaná<br>kvóta (7)..... | 22 |
| Tabulka 1  | Pole předlohy issue pro uživatelský požadavek: tři jsou povinná, tři volitelná<br>(7).....                                                                                                                                                       | 24 |